Handout #15
Thursday, 8 October 2026



Autumn Quarter 2026

Problem Set 3

Computer Files Provided for this Problem Set

You can find the following MATLAB files on the AA 279B Canvas website in the "Problem Sets and Solutions" folder:

`AA279lambert_vallado_u.m`	Vallado's Lambert solver using Universal Variables, slightly modified
`AA279jacobianshoot.m`	Sample file for computing Jacobian for shooting method

3.1) (10 points) Given the beginning (1) and ending (2) positions of an orbital transfer:

$$\underline{r_1} = \begin{bmatrix} 0 \\ 0 \\ +3 \end{bmatrix} [DU] \quad \text{and} \quad \underline{r_2} = \begin{bmatrix} 0 \\ -4 \\ 0 \end{bmatrix} [DU]$$

and assuming that this is a two body problem, what is the eccentricity of the minimum energy ellipse that contains these two points? Use canonical units, where the gravitational parameter turns out to be $\mu = 1$ $[DU^3/TU^2]$.

3.2) (25 points total) Here you'll use a shooting method to adapt a Lambert solver to handle perturbations such as a third body, oblateness (J_2), air drag, etc. For an orbital transfer near the Earth (μ_{Earth} = 398,600.4418 [km^3/sec^2]), you are given

$$\Delta t = 75\ [min] \quad \text{and} \quad \underline{r_1} = \begin{bmatrix} -15{,}000 \\ 0 \\ 0 \end{bmatrix} [km] \quad \text{and} \quad \underline{r_2} = \begin{bmatrix} -2{,}000 \\ -11{,}000 \\ 0 \end{bmatrix} [km]$$

a) (3 points) What is $\underline{v_1}$ according to the Vallado Lambert solver? Assume short-way solutions with no extra revolutions.

b) (3 points) Use MATLAB's **ode113** propagator with **options = odeset('RelTol', 1e-6, 'AbsTol', 1e-9)** to simulate a 75 [min] trajectory using $\underline{r_1}$ and $\underline{v_1}$ as initial conditions. This will require writing a simple state derivative function with 1/r^2 gravity only. Plot the trajectory in 3D to verify it really does connect $\underline{r_1}$ to $\underline{r_2}$ (you don't have to hand in this plot). Compare the simulated value of $\underline{r}$ at the end of the transfer with the desired value $\underline{r_2}$ and express the answer as an error vector. Did the Lambert solver do a good job?

c) (3 points) Update your state derivative function to include an additional perturbing oblate gravity field with oblateness coefficient J_{2earth} = 1.0826267x10^{-3} and mean equatorial radius R_{earth} = 6,378.137 [km]. The following lines of MATLAB code compute that perturbation:

```
% Acceleration due to oblateness
normr = norm(r);
rhat = r/normr;
khat = [0 0 1]';
z = r(3);
accJ2 = - (mu*J2*requatorial*requatorial/2)* ...
       ((6*z/(normr^5))*khat ...
      + ((3/(normr^4)) - (15*z*z/(normr^6)))*rhat);

% NOTE: (...) continues statement to next line
```

Rerun the 75 [min] simulation with oblateness included. Use the $\underline{v_1}$ found in part (a) in your initial conditions. Compare the simulated value of $\underline{r}$ at the end of the transfer with the desired value $\underline{r_2}$ and express the answer as an error vector.

d) (10 points) Assume that you want all the error components of the "miss distance" to be less than 10 [cm] (admittedly, this is probably overkill). Use a shooting method to perform one update step based on the error found in part (c). What is the updated $\underline{v_1}$? HINTS: You can use the provided **`AA279jacobianshoot.m`** function (be sure to read the comments) or one of your own to evaluate the Jacobian using finite differences. A small velocity increment of 0.01 [km/sec] or 0.001 [km/sec] is a reasonable place to start in numerically evaluating the Jacobian.

e) (3 points) Run the 75 [min] simulation with the updated value of $\underline{v_1}$. What is the new position error vector at the end?

f) (3 points) How many iterations are required to obtain a position error with all components less than 10 [cm]? What is the final value of $\underline{v_1}$ you obtained with the shooting method?

3.3) (65 points total) In this problem you'll analyze ΔV requirements for a system that fires a launch vehicle from a fixed launcher to intercept or rendezvous with a satellite already in orbit, mirroring what we did in class and the analysis in Section 5.7 of Bate, Mueller, and White. Along the way, you'll use some of the computer code you've used for previous space mechanics classes. (Specifically, you'll need to use Greenwich Sidereal Time to properly orient the Earth to find launcher position and velocity, and you'll need to be able to solve Kepler's equation to find a satellite's position and velocity at any time given its orbital elements.)

In November 2013, Planet Labs (based in San Francisco, CA) successfully orbited two cubesats (Dove 3 and Dove 4) from a Ukrainian Dnepr rocket. Pretend that in early 2014 one of Planet Labs' rivals, Skybox Imaging (then based in Mountain View, CA, later acquired by Google, renamed Terra Bella, and even later acquired by Planet!) decided it would like to either destroy or retrieve Planet Labs' Dove 3 satellite for competitive analysis. Imagine you were one of Skybox's interns in early 2014 and that you were assigned the task of determining ΔV tradeoffs for a launch system that could intercept or rendezvous with Dove 3.

Earth Data

Assume the Earth is a sphere (i.e. no oblateness), both when modeling its gravity field and when translating latitude/longitude/elevation to ECEF coordinates

μ_{Earth} = 398600.4418 [km^3/sec^2] gravitational parameter
r_{Earth} = 6378.137 [km] radius
ω_{Earth} = 0.0000729211585530 [rad/sec] rotation rate in inertial space

Greenwich sidereal time θ_g at the beginning of 2014 (0000h 1 January 2014 UTC) was 6.7045617 [sidereal hours].

Launcher Data

The launcher is located at Skybox's headquarters in Mountain View, CA:

longitude = -122.081944 [deg] (Western hemisphere)
latitude = +37.389444 [deg] (Northern hemisphere)
elevation above Earth sphere = 11 [m]

Satellite Data

The following is a set of orbital elements for Dove 3 in early 2014:

Year of epoch = 2014
t_0 = 21.68857885 [days] Time of epoch as day of year plus fraction (t_0 is in late-January)
i = 97.7791 [deg]
Ω = 95.3093 [deg]
e = 0.0155616
ω = 357.8679 [deg]
M_0 = 2.1812 [deg]
n = 14.56715041 [rev/day]

NOTE: t_0 is expressed in this format as "day of year plus fraction." For example, noon on January 1st would be expressed as 1.5 [days], not 0.5 [days]. Keep this in mind when writing your code…

Simulation Period

Your simulation should start at 0100h on 23 January 2014 UTC. Search a design space of two variables. The first variable is the delay between the simulation start time and the launch, i.e. "launch time." The second variable is the "time of flight" (TOF) of the launch vehicle. The first variable should range from 0 [hr] to 6 [hr], and the second should range from 0 [hr] to 2 [hr].

Additional Items

- Use short way solutions to the Lambert Problem and no multiple revolutions.
- Assume the rocket is launched directly off the surface of the earth with no gravity drag or atmospheric drag. A ΔV is applied at the launcher, and the launch vehicle flies ballistically from there.
- As discussed in lecture, before each Lambert evaluation, check that the requested transfer time doesn't force a parabolic or hyperbolic path. If it does, don't solve the Lambert Problem; just flag that point as being rejected.
- Use Vallado's algorithm discussed in lecture to flag any Lambert solutions that result in the launch vehicle's hitting the Earth before arriving at the target.
- To reduce debugging time, you may want to run on a very sparse grid at first and then move to denser and denser grids.
- If you are having trouble getting parts (a) and (b) working, we can provide some AA 279A notes that may help.

a) (3 points) Write code that computes position and inertial velocity of the launcher at a given time in ECI coordinates.

NOTE: This part requires you to find a rotation matrix for converting between Earth Centered Earth Fixed (ECEF) and Earth Centered Inertial (ECI) coordinates, which will in turn require computing the Greenwich sidereal time θ_g at any given time. In AA 279A or its equivalent, you may have done this using website lookups and/or detailed computations involving Modified Julian Date, DUT1, and UT1. For this problem set, we're asking you to do something much simpler. To find the θ_g at a given time, use

$$\theta_g(t) = \theta_g(t_{ref}) + \omega_{Earth}\,(t - t_{ref})$$

where

$\theta_g(t)$ is in [rad] .

$\theta_g(t_{ref})$ is the reference value we gave you for 0000h 1 January 2014 and needs to be converted from [sidereal hours] to [rad] using 24 [sidereal hours] = 2π [rad] . Don't forget that "sidereal hours" are simply a unit of angle measurement.

ω_{Earth} is in [rad/sec] , and

(t - t_{ref}) is the number of seconds between 0000h 1 January 2014 and the launch time t .

b) (6 points) Write code that computes position and inertial velocity of a satellite at a given time in ECI coordinates given its orbital elements.

c) (20 points) For one-burn intercepts, create a total ΔV plot (MATLAB's **`contour`** or **`contourf`** functions may help) with launch time on the horizontal axis, and time of flight (TOF) on the vertical axis. Use a grid spacing of 2 [min] or tighter. As you're determining ΔV for each combination, you can flag parabolic/hyberbolic and Earth-intersecting cases with unique/non-physical values in the matrix where you store your ΔV data. Briefly note any interesting features of the intercept ΔV plot.

d) (20 points) Create a total ΔV plot for two-burn rendezvous. Note that this data can be generated and stored inside the same loop that generates the intercept data. Briefly note any interesting features of the rendezvous ΔV plot.

e) (4 points) Within the plotted data, what launch time/TOF combination for intercept minimizes total ΔV? What is this minimum ΔV? Is it possible for this minimum value to be less than a typical circular velocity for LEO?

f) (4 points) Within the plotted data, what launch time/TOF combination for rendezvous minimizes total ΔV? What is this minimum ΔV?

g) (4 points) Assume now that the system is limited to a maximum total ΔV of 5 [km/sec]. Within the plotted data for *intercept only*, what is the earliest possible intercept time (i.e. time after start of simulation)? When does the launch occur for this case?

h) (4 points) Assume again that the system is limited to a maximum total ΔV of 5 [km/sec]. Within the plotted data for *intercept only*, what is the shortest possible time of flight? When does the launch occur for this case?